

پروژه دوم درس سیستم عامل

امیرحسین صباحی - 810101556

گزارش کار قسمت دوم: تحلیل و مقایسه عملکرد انواع Pipe

پرسش 1:

پایپ‌ها یک کانال ارتباطی یک‌طرفه بین دو نقطه ایجاد می‌کنند که در آن، داده از یک فرایند به‌عنوان «فرستنده» وارد پایپ شده و توسط فرایند دیگر به‌عنوان «گیرنده» از آن خوانده می‌شود. از منظر هسته سیستم‌عامل، پایپ‌ها در واقع یک بافر در حافظه کرنل به‌همراه دو انتها (سر خواندن و سر نوشتن) هستند. فرایندها فقط با این دو انتها (فایل‌دسکریپتورهای read و write) کار می‌کنند و مدیریت حافظه و همگام‌سازی در داخل کرنل انجام می‌شود.

در هر دو نوع پایپ (نامدار و بی‌نام) از یک مکانیزم مشترک در کرنل استفاده می‌شود: یک بافر در حافظه هسته که داده‌های نوشته‌شده توسط یک فرایند را ذخیره کرده و همان داده‌ها را در سمت دیگر توسط فرایند گیرنده در اختیار می‌گذارد. خواندن و نوشتن در هر دو نوع از طریق سیستم‌کال‌های معمول read و write روی فایل‌دسکریپتورهای مربوطه انجام می‌شود و سیاست‌های همگام‌سازی (بلوک شدن خواننده یا نویسنده) توسط کرنل مدیریت می‌گردد.

تفاوت در نحوه ایجاد:

پایپ بی‌نام با تابع `pipe()` و بدون هیچ نامی در فایل‌سیستم ایجاد می‌شود و مستقیماً یک جفت فایل‌دسکریپتور به فرایند می‌دهد.

پایپ نامدار با `mkfifo` ایجاد می‌شود و به‌صورت یک فایل ویژه نوع FIFO در فایل‌سیستم (مثلاً `/tmp/np_fifo`) ظاهر می‌گردد. فرایندها سپس با فراخوانی `open()` روی این مسیر، به پایپ متصل می‌شوند.

تفاوت در محل نگهداری داده‌ها:

در هر دو نوع پایپ، خود داده‌ها در دیسک ذخیره نمی‌شوند، بلکه در بافرهای داخلی کرنل برای pipe نگهداری می‌شوند.

تفاوت در این است که پایپ نامدار یک ورودی در فایل سیستم (inode) دارد که نقش معرف و مسیر برای اتصال فرایندها را بازی می‌کند، در حالی که پایپ بی‌نام هیچ ورودی معادل در فایل سیستم ندارد و تنها از طریق فایل دسکریپتورهای درون فرایندها قابل دسترسی است.

تفاوت در نحوه اتصال فرایندها:

در پایپ بی‌نام، اتصال معمولاً بین فرایندهایی است که از یک والد مشترک به وجود آمده‌اند؛ والد ابتدا () pipe را فراخوانی می‌کند، سپس با () fork فرایندهای فرزند را ایجاد کرده و فایل دسکریپتورهای پایپ را بین آن‌ها توزیع می‌کند. فرایندهای کاملاً مستقل، به‌طور معمول امکان دسترسی به همان unnamed pipe را ندارند. در پایپ نامدار، چون یک نام مشترک در فایل سیستم وجود دارد، هر فرایندی که مسیر FIFO را بشناسد و مجوز لازم را داشته باشد می‌تواند با () open به آن متصل شود. بنابراین این نوع پایپ برای ارتباط بین فرایندهای مستقل و بدون رابطه والد-فرزندی مناسب است.

در نتیجه، پایپ بی‌نام ابزار اصلی برای ساختن pipeline های موقتی بین فرایندهای حاصل از یک شل یا یک والد مشترک است، در حالی که پایپ نامدار نقش یک کانال نام‌گذاری شده و پایدارتر را دارد که امکان ارتباط بین فرایندهای جداگانه و در زمان‌های متفاوت را فراهم می‌سازد؛ هرچند در هر دو حالت، انتقال واقعی داده‌ها همواره از طریق بافرهای درون کرنل و مکانیزم pipe انجام می‌گیرد.

پرسش 2:

در نگاه کلی، تفاوت اصلی عملکردی بین پایپ‌های نامدار و بی‌نام بیش‌تر در مرحله «ایجاد و اتصال» آن‌هاست، نه در خود مسیر انتقال داده؛ زیرا در نهایت هر دو نوع پایپ از همان مکانیزم pipe در کرنل و بافرهای حافظه‌ای مشترک استفاده می‌کنند. با این حال چند عامل در سطح سیستم (مانند تعداد و نوع فراخوانی‌های سیستمی، نیاز به دسترسی به فایل سیستم و الگوی اتصال فرایندها) می‌تواند باعث اختلاف در کارایی شود.

از نظر فراخوانی‌های سیستمی و بهره‌وری CPU، پایپ بی‌نام معمولاً در سناریوی متعارف به‌این‌شکل استفاده می‌شود: ابتدا تابع pipe فراخوانی می‌شود، سپس با یک یا چند بار fork فرایندهای فرزند ایجاد می‌شوند و با dup2 ورودی و خروجی استاندارد آن‌ها به دو انتهای پایپ متصل می‌گردد و در نهایت execve برای اجرای برنامه‌ها صدا زده می‌شود. یعنی زنجیره‌ای از فراخوانی‌های pipe، fork، dup2 و execve.

در مقابل، پایپ نامدار ابتدا با mkfifo در فایل سیستم ساخته می شود و هر فرایندی که بخواهد به آن متصل شود باید با فراخوانی open روی مسیر مربوطه آن را باز کند و سپس با read و write روی همان فایل دسکریپتور کار کند. بنابراین برای هر اتصال جدید به FIFO حداقل یک open و یک close اضافه بر خواندن و نوشتن داریم و این یعنی مقداری سربار بیشتر در تعداد فراخوانی های سیستمی و صرف زمان CPU، به خصوص در سناریوهایی که اتصال ها کوتاه عمر و پرتعداد هستند. با این وجود، در بخش انتقال واقعی داده (حلقه های read/write روی pipe)، هزینه پردازشی در هر دو نوع تقریباً یکسان است، زیرا هر دو به پیاده سازی مشترک pipe در کرنل متکی هستند.

از نظر دسترسی به فایل سیستم، پایپ بی نام هیچ نامی در فضای فایل سیستم ندارد و کرنل فقط یک شی pipe به همراه دو فایل دسکریپتور را نگه می دارد؛ فرایندها بدون هیچ عملیات نام گذاری، صرفاً از طریق این fd ها به پایپ دسترسی دارند. در مقابل، پایپ نامدار به صورت یک فایل ویژه از نوع FIFO در فایل سیستم ظاهر می شود. هر بار که فرایندی با open("path", ...) به آن متصل می شود، باید مسیر در لایه VFS تحلیل شود، inode متناظر پیدا گردد و مجوزها و سیاست های امنیتی بررسی شود. این عملیات، هرچند سریع و cache محور است، اما نسبت به unnamed pipe که چنین مرحله ای ندارد، یک هزینه اضافی در راه اندازی ارتباط محسوب می شود. بنابراین در سناریوهایی که تعداد اتصال و قطع (open/close) زیاد است، این سربار فایل سیستمی می تواند تأثیر محسوسی بر زمان اولیه اتصال و بهره وری CPU داشته باشد؛ در حالی که در ارتباط های طولانی مدت، نسبت به کل زمان اجرا معمولاً ناچیز است.

از منظر زمان بندی و context switch، هر دو نوع پایپ رفتار بسیار مشابهی دارند. هر دو از همان مکانیزم بلوکه شدن خواننده در صورت خالی بودن بافر و بلوکه شدن نویسنده در صورت پر بودن بافر استفاده می کنند و بیدار کردن فرایندها بر عهده کرنل است. تفاوت جدی ساختاری در سطح مکانیزم زمان بندی بین named و unnamed وجود ندارد و هر اختلافی بیشتر به الگوی استفاده بستگی دارد؛ برای مثال، یک pipeline کوتاه مدت با چند فرایند هم زمان (در پایپ بی نام) نسبت به یک daemon طولانی مدت که همیشه روی یک FIFO منتظر داده است (در پایپ نامدار) می تواند الگوی context switch متفاوتی در عمل ایجاد کند، اما این اختلاف به «نوع پایپ» کمتر و به «سناریوی برنامه نویسی» بیشتر وابسته است.

در جمع بندی عملکردی می توان گفت پایپ بی نام برای ارتباط های سریع، کوتاه مدت و بین فرایندهای مرتبط (مانند پیاده سازی a | b | c در یک شل) معمولاً کارا تر است، زیرا نیازی به دسترسی به فایل سیستم و فراخوانی open روی مسیر ندارد و با تعداد کمتری از فراخوانی های سیستمی و منطق ساده تر راه اندازی می شود. در مقابل، پایپ نامدار برای ارتباط های بلند مدت یا بین فرایندهای مستقل و نامرتبط که ممکن است در زمان های مختلف اجرا شوند مناسب تر است؛ اگرچه برای ایجاد و اتصال، هزینه بیشتری به صورت فراخوانی mkfifo و open و عبور از لایه فایل سیستم پرداخت می شود، اما این هزینه در برابر مزیت داشتن یک نام ثابت و نقطه اتصال مشترک در بسیاری از کاربردها پذیرفتنی است. در هر دو حالت، تفاوت عملکردی از تفاوت در نوع بافر یا شیوه انتقال داده در کرنل ناشی نمی شود، بلکه بیشتر به تفاوت در تعداد و نوع فراخوانی های سیستمی، نیاز یا عدم نیاز به دسترسی

به فایل سیستم، و الگوهای زمانی اجرای فرایندها برمی گردد که می تواند بر مصرف CPU، زمان تأخیر اولیه اتصال و رفتار کلی سیستم اثر بگذارد.

پرسش 3:

در سطح کاربرد، می توان گفت پایپ بی نام و پایپ نامدار هر کدام برای سناریوهای متفاوتی طراحی شده اند و انتخاب بین آن ها وابسته به این است که فرایندها چه نسبتی با هم دارند، ارتباط چقدر طول می کشد و چقدر نیاز به انعطاف در اتصال فرایندها داریم.

استفاده از پایپ بی نام زمانی مناسب تر است که ارتباط بین فرایندها کوتاه مدت، خطی و در یک زنجیره مشخص باشد؛ در این حالت یک فرایند والد (مثلاً شل) با pipe و fork فرزندان را ایجاد می کند، به سادگی ورودی و خروجی آن ها را همگام می کند و بعد از پایان اجرا، هم فرایندها و هم پایپ به طور خودکار از بین می روند. این رویکرد عملکرد بهتری دارد، زیرا نیازی به نام گذاری در فایل سیستم و عملیات اضافی open ندارد و تعداد فراخوانی های سیستمی و سر بار مدیریتی آن کم تر است.

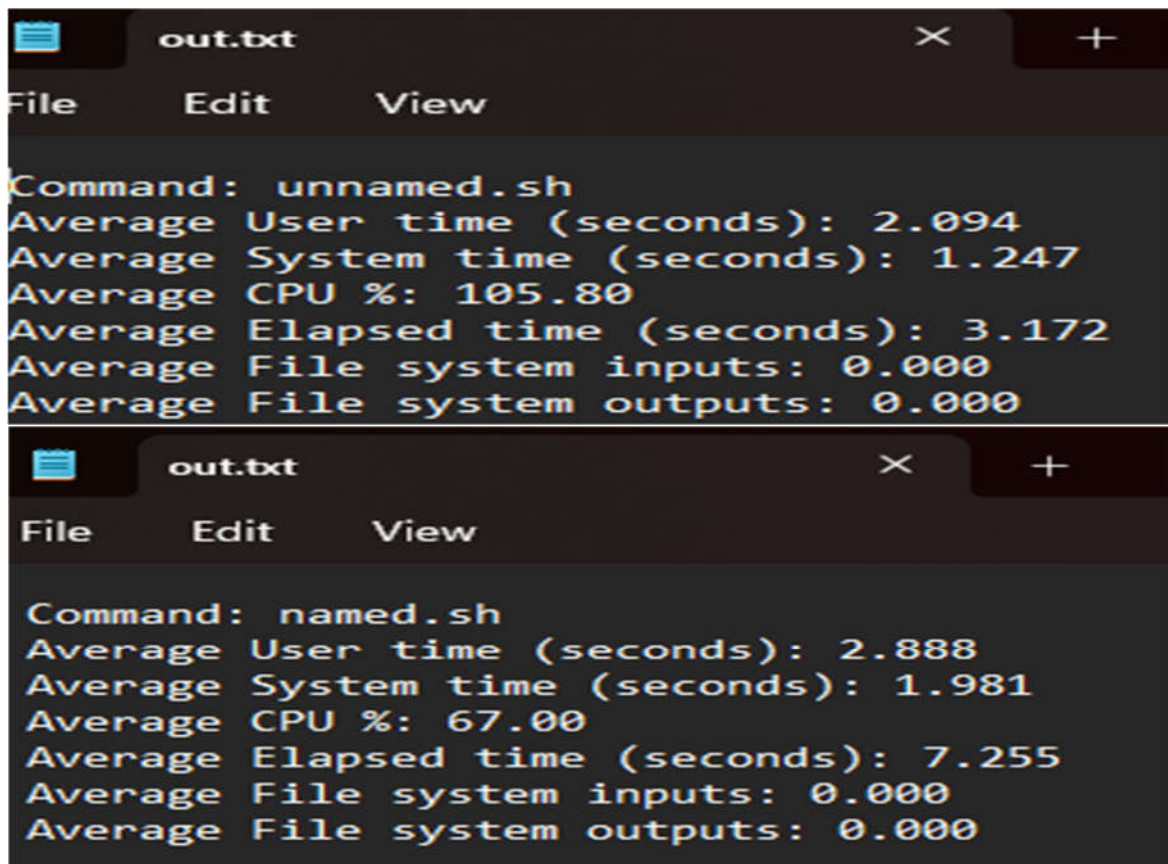
در مقابل، پایپ نامدار زمانی مناسب تر است که نیاز به ارتباط بین فرایندهای مستقل و نامرتب داریم که الزاماً از یک والد مشترک نیامده اند یا حتی در زمان های مختلف اجرا می شوند؛ مثلاً یک سرویس دائمی که همیشه از یک FIFO می خواند و کلاینت های مختلف هر زمان لازم شد روی همان FIFO می نویسند. در این حالت نامدار بودن پایپ (وجود یک مسیر مشخص در فایل سیستم) انعطاف پذیری بالاتری می دهد، زیرا فرایندها فقط با دانستن آن نام می توانند به کانال ارتباطی مشترک متصل شوند، حتی اگر هیچ رابطه fork با هم نداشته باشند.

از نظر مصالحه ای میان انعطاف پذیری، همگام سازی، طول عمر و عملکرد، می توان گفت پایپ بی نام در همگام سازی ساده و خودکار یک زنجیره محدود از فرایندها، با عمر کوتاه و عملکرد بالا، انتخاب طبیعی است؛ ولی در عوض انعطاف پذیری کمتری دارد و به راحتی برای فرایندهای مستقل قابل استفاده نیست. پایپ نامدار انعطاف پذیری بسیار بیشتری ارائه می کند و اجازه می دهد کانال های ارتباطی پایدار و چندفرایندی طراحی شود، اما در مقابل هزینه بیشتری برای ایجاد و اتصال (به دلیل دسترسی به فایل سیستم و فراخوانی های open/close) می پردازد و طول عمرش از فرایندها جدا می شود؛ یعنی باید به صورت صریح ایجاد و حذف شود. به طور خلاصه، هر چه به سمت ارتباط های

کوتاه‌مدت، متمرکز و ساده‌تر برویم، پایپ بی‌نام از نظر عملکرد و سادگی برتری دارد؛ و هر چه به سمت ارتباط‌های بلندمدت، پویا و بین فرایندهای مستقل برویم، پایپ نامدار با وجود سربار بیشتر، از نظر طراحی و انعطاف‌پذیری گزینه مناسب‌تری است.

مقایسه عملی پایپ نامدار و بینام

```
oteook@Amirhosein:/mnt/f/Term7/OS/OS-CA2$ bash benchmark.sh named.sh out.txt
Running named.sh 10 times and collecting statistics...
Run 1...
Run 2...
Run 3...
Run 4...
Run 5...
Run 6...
Run 7...
Run 8...
Run 9...
Run 10...
Averages written to out.txt
oteook@Amirhosein:/mnt/f/Term7/OS/OS-CA2$ bash benchmark.sh unnamed.sh out.txt
Running unnamed.sh 10 times and collecting statistics...
Run 1...
Run 2...
Run 3...
Run 4...
Run 5...
Run 6...
Run 7...
Run 8...
Run 9...
Run 10...
Averages written to out.txt
```



The image shows two screenshots of a terminal window. The top screenshot displays the output for the command 'unnamed.sh', showing metrics such as Average User time (2.094s), Average System time (1.247s), Average CPU % (105.80), Average Elapsed time (3.172s), Average File system inputs (0.000), and Average File system outputs (0.000). The bottom screenshot displays the output for the command 'named.sh', showing metrics such as Average User time (2.888s), Average System time (1.981s), Average CPU % (67.00), Average Elapsed time (7.255s), Average File system inputs (0.000), and Average File system outputs (0.000).

```
out.txt
File Edit View
Command: unnamed.sh
Average User time (seconds): 2.094
Average System time (seconds): 1.247
Average CPU %: 105.80
Average Elapsed time (seconds): 3.172
Average File system inputs: 0.000
Average File system outputs: 0.000

out.txt
File Edit View
Command: named.sh
Average User time (seconds): 2.888
Average System time (seconds): 1.981
Average CPU %: 67.00
Average Elapsed time (seconds): 7.255
Average File system inputs: 0.000
Average File system outputs: 0.000
```

اول از همه می‌شود دید که مجموع زمان پردازنده یعنی جمع user و system برای اسکریپت مبتنی بر پایپ نامدار حدوداً ۴٫۸۸ ثانیه است، در حالی که همین مقدار برای اسکریپت مبتنی بر پایپ بی‌نام حدود ۳٫۳۴ ثانیه است. بنابراین خودِ استفاده از named pipe تقریباً یک و نیم ثانیه زمان CPU بیش‌تر مصرف کرده است. این اختلاف عمدتاً در مؤلفه‌ی system time دیده می‌شود: در named.sh حدود ۱٫۹۸۱ ثانیه و در unnamed.sh حدود ۱٫۲۴۷ ثانیه. این همان چیزی است که در سؤال تشریحی دوم هم پیش‌بینی شده بود؛ یعنی پایپ نامدار به دلیل نیاز به فراخوانی‌های سیستمی اضافی مانند mkfifo، open و unlink و عبور از لایه‌ی فایل سیستم برای resolve کردن مسیر /tmp/f و بررسی مجوزها، سربار سیستمی بیش‌تری نسبت به پایپ بی‌نام دارد. در مقابل، در unnamed.sh ایجاد و اتصال پایپ فقط با pipe، fork، dup2 و exec انجام می‌شود و نیازی به name resolution در فایل سیستم نیست، بنابراین زمان system کم‌تر است. زمان سپری‌شده‌ی واقعی یا elapsed نیز همین الگو را تأیید می‌کند. در named.sh، متوسط elapsed حدود ۷٫۲۵ ثانیه و در unnamed.sh حدود ۳٫۱۷ ثانیه است؛ یعنی اسکریپتی که از پایپ بی‌نام استفاده می‌کند تقریباً با ضریب حدود دو برابر سریع‌تر تمام می‌شود. در واقع، در named.sh بخش قابل توجهی از این زمان صرف انتظار برای عملیات روی FIFO می‌شود؛ از جمله ایجاد و حذف مکرر FIFO در هر تکرار و همچنین semantics خاص open روی FIFO که ممکن است باعث بلوک شدن فرایند تا حضور طرف مقابل شود. در

unnamed.sh، ارتباط بین dd و cat ساده‌تر و مستقیم‌تر است و به محض ایجاد پایپ و راه‌اندازی دو فرایند، انتقال داده انجام می‌شود و حلقه سریع‌تر پیش می‌رود.

درصد استفاده از CPU نیز این تفاوت را تأیید می‌کند. در named.sh، CPU% حدود ۶۷٪ است؛ یعنی نسبت زمان پردازنده به زمان سپری‌شده کمتر است و بخش قابل توجهی از elapsed صرف انتظار برای I/O یا همگام‌سازی روی FIFO شده است. در unnamed.sh، CPU% حدود ۱۰۵٫۸٪ گزارش شده است که به‌خاطر اجرای هم‌زمان چند فرایند مثلاً dd و cat روی چند هسته است؛ مجموع زمان پردازنده نسبت به elapsed بیش‌تر از صد درصد می‌شود و نشان می‌دهد که سیستم در آن بازه نسبتاً پرکارتر بوده و زمان کم‌تری صرف انتظار شده است.

بنابراین، مقایسه‌ی مقادیر user، system، elapsed و CPU% نشان می‌دهد که اسکرپت مبتنی بر پایپ بی‌نام هم از نظر زمان کل پردازنده و هم از نظر زمان سپری‌شده کارا تر است و اسکرپت مبتنی بر پایپ نامدار هزینه‌ی اضافی قابل توجهی به سیستم تحمیل می‌کند. این نتایج با انتظارات بیان‌شده در پاسخ سؤال تشریحی دوم هم‌خوانی دارد؛ آن‌جا توضیح داده شد که تفاوت اصلی در عملکرد دو نوع پایپ، نه در مکانیزم انتقال داده در کرنل (که در هر دو مورد مشترک است)، بلکه در تعداد و نوع فراخوانی‌های سیستمی، لزوم دسترسی به فایل سیستم و نحوه‌ی اتصال فرایندهاست. در این آزمایش عملی نیز دقیقاً مشاهده می‌شود که پایپ نامدار به دلیل همین هزینه‌های اضافی، system time و elapsed بیشتری نسبت به پایپ بی‌نام دارد، در حالی که پایپ بی‌نام با سر بار کمتر، عملکرد بهتری از خود نشان می‌دهد.